

THE PARALLEL TERMINALS PLAYBOOK

How to Run Multiple Claude Code Sessions Without Burning Money

By Mark Kashef
Early AI Adopters

The Fantasy vs Reality

You've seen the screenshots on LinkedIn and X. Someone flexing 5, 10, 50, even hundreds of Claude Code terminals running simultaneously.

Looks impressive. Until you zoom in.

What You'll Actually See:

- Many terminals asking basic questions that could be answered in 30 seconds
- Sessions colliding with each other, stepping on each other's toes
- Features being built that are so brittle they'll break the moment they're touched

The truth: Adding 8 terminals won't make you 8x faster. Just like adding double the developers doesn't halve development time.

Every build has nuance about whether it's even eligible to run in parallel, and what order tasks should execute if they are eligible.

The One Question That Changes Everything

Before spinning up a new terminal, ask yourself:

"Does this task depend on anything else being completed first?"

- **If YES** → It's not ready for parallel execution
- **If NO** → You might be able to run it independently

Scenario 1: True Parallel

Best for: Non-technical tasks that are genuinely independent.

This is where you can realize the fantasy of multiple terminals. Nothing bad will happen IF:

1. You know exactly what you're trying to do
2. You know the intermediary steps needed
3. You have proper prompts that don't need babysitting

Example: Launching an AI Course

Let's say you need distribution for your course. These tasks can run truly in parallel:

#	Task Prompt
1	Research the top 10 AI course competitors. List their pricing, unique angles, and target audience. Save to research_competitors.md
2	Draft a 5-email welcome sequence for new subscribers to my AI newsletter. Save to welcome_emails.md
3	Find 20 tech influencers on YouTube with 100-100K subscribers who review online courses. Save to influencers.md
4	Write landing page copy with headline, sub-headline, three bullet points, SEO optimized. Save to landing_page.md

Why these work in parallel: None of these tasks need information from the others to complete. They're truly independent.

The Gut Check

Before running tasks in parallel, ask:

- Would Task A benefit from knowing what Task B discovered?
- Could the output of one task change how I'd approach another?

If you're a beginner, you can ask Claude Code directly:

```
"Look at the rest of our plan and tell me what can be built independently - in a way where if it had no context of the other tasks, I could run it in a separate session."
```

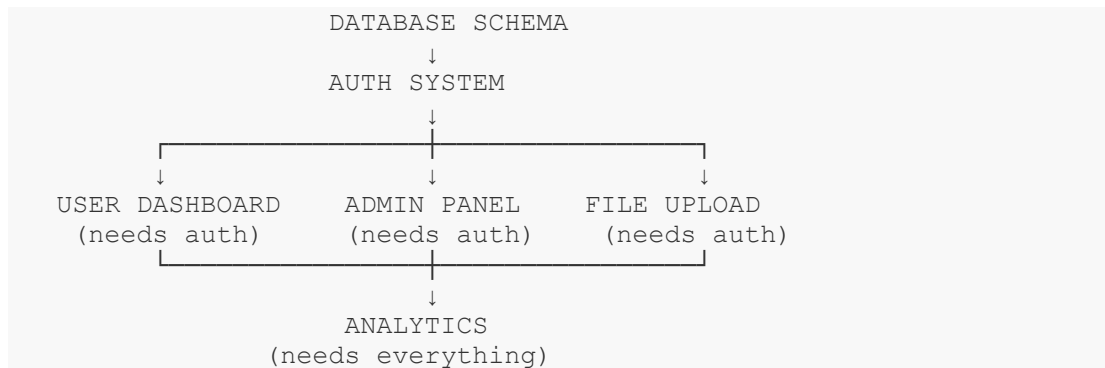
Scenario 2: Phased Parallel

Best for: Technical builds where a foundation must exist before auxiliary features.

This is where most real work happens. You build a foundation first, THEN spin up parallel sessions for features that can be independent.

The Dependency Mental Model

Think of it like building blocks. Some things **MUST** come first:



Phase Execution Example

Phase 1 (Sequential - Must Complete First)

- Set up database schema for client portal
- Build authentication system with login, signup, password reset

Phase 2 (Parallel - After Foundation)

#	Terminal Task
1	Build user dashboard showing project list and status. Auth and database already set up.
2	Build admin panel for client management. Auth and database already set up.
3	Build file upload system. Auth and database already set up.

Key: Tell each session that the foundation is already complete so it doesn't try to rebuild it.

Asking Claude for Help

If you're non-technical and don't know what's independent, use this prompt:

"We just finished the foundation. Can you look at the remaining features and tell me which ones can be built in separate sessions without knowing about each other?"

Also, write me a meta-prompt for each session that includes enough context to build independently."

Scenario 3: The Relay Race

Best for: Complex builds with clear phases where each depends on the previous.

This is like passing a baton from one session to the next.

Why This Approach Works

1. Each phase gets 100% fresh context
2. No confusion from auto-compacting
3. Clear handoffs between sessions

The Auto-Compact Problem

When you auto-compact a conversation, you don't know what Claude prioritized or deprioritized. You keep running the next session at a disadvantage because nuance gets lost.

The relay race solves this: Each session is mutually exclusive in context. Even if auto-compact isn't perfect, you have the 80/20 of what you need.

The Plan.md Approach

Create a plan file with checkboxes that Claude updates as it works:

```
# Project Plan

## Overview
[Big picture explanation of the project]

## Phase 1: Foundation
- [ ] Set up database schema
- [ ] Create authentication system
- [ ] Configure environment variables

## Phase 2: Core Features
- [ ] User dashboard
- [ ] Admin panel
- [ ] File uploads

## Phase 3: Integrations
- [ ] Payment processing
- [ ] Email notifications
```

The Bonus Session Trick

Keep one extra session open - the one where you created the plan. This session has:

- Virgin context (not polluted by implementation details)
- Full understanding of the plan
- Ability to verify other sessions' work

Use it to check in after each phase:

```
"Can you peek at phase 3? We just completed it in another
terminal. Check the codebase and tell me - have we finished
everything we need to do?"
```

Sometimes it catches things other sessions missed. This puts you on offense instead of defense.

Relay Race Prompt Templates

Starting a Phase

```
Read plan.md. Execute Phase [N]: [Phase Name].
```

As you work:

- Check off completed tasks in plan.md
- Add any architecture decisions to decisions.md
- Document anything the next session needs to know

```
When done, summarize what was completed.
```

Continuing After Previous Phase

```
Read plan.md. Phase [N-1] is complete.
```

```
Execute Phase [N]: [Phase Name].
```

```
Reference decisions.md for any architectural context
from previous phases.
```

```
Check off tasks as you complete them.
```

Quick Reference

When to Use Each Scenario

Scenario	Use When	Example
True Parallel	Tasks are genuinely independent, no shared dependencies	Research, content creation, competitor analysis
Phased Parallel	Foundation exists, auxiliary features can be independent	Building features after database/auth setup
Relay Race	Complex builds with clear sequential phases	Multi-phase projects with dependencies

The 5 Key Principles

1. **More terminals ≠ more productivity** — Without strategy, you're burning money
2. **Dependencies matter** — If Task B needs Task A's output, they can't run in parallel
3. **Fresh context is valuable** — The relay race approach keeps each session focused
4. **Plan guardians catch mistakes** — Keep one session dedicated to verifying the plan
5. **Document decisions** — When you deviate from the plan, write it down for future sessions

The One Question

Before starting any new terminal session, ask:

"Does this task depend on anything else, or can it run completely independently?"

That single question will save you hours of debugging collisions and wasted API calls.

Want More?

Join the Early AI Adopters community for:

- Full video walkthroughs of real builds
- Exclusive content that never hits YouTube
 - Direct access to me for questions
- A community of builders who are actually shipping

skool.com/earlyaidopters